DOCUMENTED WORKFLOWS

1. Executive Summary

This document outlines the development, testing, deployment, and version control workflows for the **Zupharm Laboratories Website** project. These workflows ensure code quality, team collaboration, and reliable deployments.

Current Status: Phase 1 (MVP)
Team Size: 4-6 developers
Repository: GitHub (Farha-t/SPRINT_Pharma)
Deployment: Netlify (auto-deploy on push to main)

---

2. Git Workflow & Version Control Strategy

2.1 Branching Strategy (Git Flow Modified)

```
main (production)
  ↑
  ├─ Hotfixes (critical bugs only)
  │
develop (staging/integration)
  ↑
  ├─ feature/* (new features)
  ├─ bugfix/* (bug fixes)
  └─ enhancement/* (improvements)
```

2.2 Branch Naming Conventions

Branch Type	Pattern	Example	Purpose
Main Branch	`main`	`main`	Production-ready code
Develop Branch	`develop`	`develop`	Integration & testing branch
Feature Branch	`feature/short-description`	`feature/add-blog-page`	New feature development
Bugfix Branch	`bugfix/issue-number`	`bugfix/contact-form-submit`	Bug fix for issue
Enhancement	`enhancement/description`	`enhancement/mobile-menu-animation`	Code improvement

Branch Type	Pattern	Example	Purpose
Hotfix Branch	`hotfix/critical-issue`	`hotfix/payment-gateway-error`	Critical production fix

2.3 Git Workflow Steps

Step 1: Create Feature Branch

```
# Update main/develop to latest
git checkout develop
git pull origin develop

# Create feature branch from develop
git checkout -b feature/add-product-filter

# Push branch to GitHub
git push -u origin feature/add-product-filter
```

Step 2: Local Development

```
# Work on feature locally
git status
git add src/components/FilterBar.jsx
git commit -m "feat: add product filtering component"

# Push commits regularly (at least daily)
git push origin feature/add-product-filter

# Continue making commits
git commit -m "feat: add filter state management"
git push origin feature/add-product-filter
```

Step 3: Pull Request (PR) Creation

```
PR Title: "[FEATURE] Add Product Filter Component"

Description:
## Description
Implements product filtering by category, price range, and therapeutic segment.

## Changes
- Created FilterBar.jsx component
- Added filter state to Products page
- Updated ProductGrid to accept filter props
```

```
## Testing Done
- Tested on Chrome, Firefox, Safari
- Tested mobile responsiveness (375px, 768px, 1024px)
- Verified filter persists on page refresh

## Screenshots
[Screenshot 1: Filter UI]
[Screenshot 2: Mobile view]

## Related Issue
Closes #42
```

Step 4: Code Review Process

```
1. PR submitted → status: "Needs Review"
2. Reviewer assigned (Piyush/Farhat/Senior Dev)
3. Automated checks run:
   - ESLint ✓ (code quality)
   - Build test ✓ (no build errors)
   - Accessibility check (axe-core)
4. Manual review:
   - Code logic review
   - Performance check
   - Security review
   - UX/UI consistency
5. Comments & suggestions added if needed
6. Developer addresses feedback
7. Approval given → ready to merge
```

Step 5: Merge to Develop

```
# After PR approval, merge to develop
git checkout develop
git pull origin develop
git merge feature/add-product-filter
git push origin develop

# Delete feature branch
git branch -d feature/add-product-filter
git push origin --delete feature/add-product-filter
```

Step 6: Merge Develop → Main (Release)

```
# Create release PR from develop → main
git checkout main
git pull origin main
```

```
git merge develop
git push origin main

# Netlify auto-deploys on main push
# → Builds → Deploys to production
```

2.4 Commit Message Convention

Format: `type(scope): description`

Type	Scope	Example
`feat`	Component/feature name	`feat(contact-form): add email validation`
`fix`	Component/bug name	`fix(header): resolve mobile menu toggle`
`docs`	Documentation	`docs(readme): update setup instructions`
`style`	CSS/styling	`style(colors): update teal shade for WCAG`
`refactor`	Component name	`refactor(products): extract card to component`
`perf`	Performance optimization	`perf(home): lazy load images below fold`
`test`	Test files	`test(accordion): add unit tests`
`ci`	CI/CD config	`ci(github-actions): add lint check`

Examples:

```
feat(blog): add blog search functionality
fix(franchise-form): fix submit handler validation
docs(architecture): add component hierarchy diagram
style(tailwind): adjust color palette for accessibility
refactor(header): extract navigation to separate component
perf(images): optimize product images with srcset
test(tabs): add integration tests for tab switching
ci(netlify): update build script for production
```

2.5 Git Workflow Diagram

```
│  develop (staging)                                          │
│    ├─ feature/add-product-filter                            │
│    ├─ feature/improve-animations                            │
│    ├─ bugfix/contact-form-validation                        │
│    └─ enhancement/accessibility                             │
│                                                             │
└─────────────────────────────────────────────────────────────┘
```

---

3. Development Workflow

3.1 Feature Development Lifecycle

```
Jira Ticket Created
        ↓
┌──────────────────────────────────┐
│ Issue Analysis & Estimation      │
├──────────────────────────────────┤
│ - Product owner reviews          │
│ - Team estimates story points    │
│ - Acceptance criteria defined    │
│ - Dev assigned                   │
└──────────────────────────────────┘
        ↓
┌──────────────────────────────────┐
│ Development Sprint Planning      │
├──────────────────────────────────┤
│ - 2-week sprint cycles           │
│ - Daily standup (10 mins)        │
│ - Sprint backlog defined         │
└──────────────────────────────────┘
        ↓
┌──────────────────────────────────┐
│ Local Development                │
├──────────────────────────────────┤
│ 1. Create feature branch         │
│ 2. npm install (if needed)       │
│ 3. npm run dev (start dev server)
│ 4. Code implementation           │
│ 5. Test locally (all browsers)   │
│ 6. Commit & push regularly       │
└──────────────────────────────────┘
        ↓
┌──────────────────────────────────┐
│ Code Quality Checks              │
├──────────────────────────────────┤
│ 1. npm run lint (ESLint)         │
│ 2. Manual code review            │
│ 3. Accessibility check (axe)     │
└──────────────────────────────────┘
        ↓
```

```
┌─────────────────────────────────────┐
│ Pull Request (PR) Review            │
├─────────────────────────────────────┤
│ 1. Push to GitHub                   │
│ 2. Create PR to develop             │
│ 3. Request code review              │
│ 4. Address feedback                 │
│ 5. Get approval                     │
└─────────────────────────────────────┘
        ↓
┌─────────────────────────────────────┐
│ QA Testing (Develop Branch)         │
├─────────────────────────────────────┤
│ 1. Deploy to staging                │
│ 2. QA tests feature                 │
│ 3. Report bugs (if any)             │
│ 4. Mark as ready or reject          │
└─────────────────────────────────────┘
        ↓
┌─────────────────────────────────────┐
│ Merge to Main & Deploy              │
├─────────────────────────────────────┤
│ 1. Merge develop → main             │
│ 2. Netlify auto-builds              │
│ 3. Tests run in CI/CD               │
│ 4. Deploy to production             │
│ 5. Monitor for errors               │
└─────────────────────────────────────┘
        ↓
┌─────────────────────────────────────┐
│ Production Release                  │
├─────────────────────────────────────┤
│ 1. Feature live for users           │
│ 2. Monitor analytics                │
│ 3. Gather user feedback             │
│ 4. Close Jira ticket                │
│ 5. Retrospective notes              │
└─────────────────────────────────────┘
```

3.2 Daily Development Routine

Morning (9:00 AM):

```
1. Check Slack for blockers
2. Review overnight CI/CD results
3. Sync feature branch with develop
   git pull origin develop
   git rebase develop
4. Check PR review comments
5. Address feedback/resolve conflicts
```

During Day (10:00 AM - 5:00 PM):

```
1. Code feature implementation
2. Test locally: npm run dev
3. Commit changes regularly
   git add .
   git commit -m "feat(name): description"
   git push origin feature/xxx
4. Respond to PR comments
5. Help team members with blockers
```

End of Day (5:00 PM):

```
1. Ensure latest code is pushed
   git push origin feature/xxx
2. Update Jira ticket status
3. Document progress in PR
4. Note any blockers for next day
```

3.3 Code Quality Standards

ESLint Rules Enforced:

```
// ✅ Good
const handleClick = () => { /* ... */ };
const items = array.map(item => item.value);

// ❌ Bad
var handleClick = function() { /* ... */ };
const items = array.map(function(item) { return item.value; });
```

Naming Conventions:

```
✅ Good:
- Components: PascalCase (ProductCard.jsx)
- Functions: camelCase (handleSubmit)
- Constants: UPPER_SNAKE_CASE (MAX_ITEMS)
- CSS Classes: kebab-case (product-card)
- Files: PascalCase (Home.jsx)

❌ Bad:
- productCard.jsx
- HandleSubmit
- maxItems
```

```
- product_card
- home.jsx
```

Component Structure:

```
// ✅ Recommended Structure
import React, { useState } from 'react';
import PropTypes from 'prop-types';
import './Component.css';

const MyComponent = ({ prop1, prop2 }) => {
  const [state, setState] = useState(null);

  const handleAction = () => { /* ... */ };

  return (
    <div className="component">
      {/* JSX */}
    </div>
  );
};

MyComponent.propTypes = {
  prop1: PropTypes.string.isRequired,
  prop2: PropTypes.number,
};

export default MyComponent;
```

---

4. QA Testing Workflow

4.1 QA Testing Phases

```
Development Environment (Local Dev)
        ↓
┌─────────────────────────────────┐
│ Developer Self-Testing          │
├─────────────────────────────────┤
│ - Desktop (Chrome, Firefox)     │
│ - Mobile (375px, 768px)         │
│ - Forms & inputs                │
│ - Navigation & routing          │
│ - Performance (Lighthouse)      │
│ - Accessibility (axe)           │
└─────────────────────────────────┘
        ↓
Staging Environment (Develop Branch)
        ↓
```

```
┌─────────────────────────────────────┐
│ QA Tester Manual Testing            │
├─────────────────────────────────────┤
│ - Feature functionality             │
│ - Cross-browser testing             │
│ - Responsive design check           │
│ - User workflows                    │
│ - Edge cases & error handling       │
└─────────────────────────────────────┘
        ↓
Production Environment (Main Branch)
        ↓
┌─────────────────────────────────────┐
│ Smoke Testing (Post-Deploy)         │
├─────────────────────────────────────┤
│ - Critical paths work               │
│ - Forms submit correctly            │
│ - No console errors                 │
│ - Analytics tracking                │
└─────────────────────────────────────┘
```

4.2 Test Case Examples

Test Case: Contact Form Submission

```
Test ID: TC-001
Component: Contact.jsx
Description: User submits contact form

Preconditions:
- User navigates to /contact
- Browser: Chrome (latest)
- Device: Desktop

Steps:
1. User enters name: "John Doe"
2. User enters email: "john@example.com"
3. User enters phone: "+91 9876543210"
4. User selects inquiry type: "PCD Franchise Application"
5. User enters message: "Interested in franchise"
6. User clicks "Submit"

Expected Results:
✓ Form is validated
✓ No console errors
✓ Success message displays
✓ Form is cleared
✓ Data sent to backend API
✓ User sees confirmation message

Actual Results:
```

```
[QA Tester fills this in]

Status: ☐ Pass  ☐ Fail  ☐ Blocked
```

Test Case: Product Filter

```
Test ID: TC-002
Component: Products.jsx + FilterBar.jsx
Description: User filters products by category

Preconditions:
- User navigates to /products
- All 12 product categories visible

Steps:
1. User sees all 12 product categories
2. User clicks "Cardiac Care" category
3. Application filters to show only Cardiac products

Expected Results:
✓ Filter applied instantly
✓ Product grid updates
✓ Filter state persists on reload
✓ "Clear Filter" button appears
✓ Mobile view works

Actual Results:
[QA Tester fills this in]

Status: ☐ Pass  ☐ Fail  ☐ Blocked
```

4.3 Bug Logging Process

When QA Finds a Bug:

```
1. Document the Bug:
   - Title: [BRIEF DESCRIPTION]
   - Component: Which page/component
   - Severity: Critical | High | Medium | Low
   - Steps to reproduce
   - Expected vs actual behavior
   - Screenshots/recordings
   - Browser & device info

2. Log in Jira:
   - Project: SPRINT_PHARMA
   - Issue Type: Bug
   - Assignee: Relevant developer
   - Priority: Blocker | High | Medium | Low
```

```
3. Communication:
   - Post in #qa-bugs Slack channel
   - Tag affected developer
   - Link Jira ticket

4. Developer Fixes:
   - Create bugfix/* branch
   - Make fixes
   - Re-test with QA
   - PR → Code Review → Merge

5. QA Verification:
   - QA re-tests on develop
   - Mark as verified/fixed
   - Close Jira ticket
```

4.4 Testing Checklist

Pre-Deployment Checklist:

```
☐ All tests pass (npm run test)
☐ No console errors in DevTools
☐ Lighthouse score > 90 (performance)
☐ Accessibility score > 95
☐ Form submissions work
☐ Navigation works all routes
☐ Mobile responsive (375px, 768px, 1024px)
☐ Cross-browser tested (Chrome, Firefox, Safari, Edge)
☐ Images load correctly
☐ Links go to correct pages
☐ Contact forms send data
☐ Blog pages load correctly
☐ Animation performance smooth (60 FPS)
☐ SEO metadata present
☐ Social meta tags correct
```

---

5. Build & Deployment Workflow

5.1 Build Process

```
Developer Pushes to Main
        ↓
┌─────────────────────────────────────┐
│ GitHub Webhook Triggered            │
├─────────────────────────────────────┤
│ - Netlify receives notification     │
│ - Build starts automatically        │
```

```
└──────────────────────────────────┘
         ↓
┌──────────────────────────────────┐
│ Build Environment (Netlify)      │
├──────────────────────────────────┤
│ 1. Checkout code from main       │
│ 2. Node.js installed (v16+)      │
│ 3. npm install dependencies      │
│ 4. Build command runs:           │
│    npm run build                 │
│ 5. Output: dist/ folder          │
└──────────────────────────────────┘
         ↓
┌──────────────────────────────────┐
│ Optimization & Checks            │
├──────────────────────────────────┤
│ 1. Minify JS/CSS                 │
│ 2. Compress images               │
│ 3. Tree shake unused code        │
│ 4. Generate source maps          │
└──────────────────────────────────┘
         ↓
┌──────────────────────────────────┐
│ Deployment to CDN                │
├──────────────────────────────────┤
│ 1. Upload dist/ to Netlify Edge  │
│ 2. Replicate to global CDN       │
│ 3. Set cache headers             │
│ 4. Purge old cache               │
└──────────────────────────────────┘
         ↓
┌──────────────────────────────────┐
│ Post-Deploy Tests                │
├──────────────────────────────────┤
│ 1. Health check: GET /           │
│ 2. Status: 200 OK                │
│ 3. No 404 errors                 │
│ 4. Smoke test: all routes load   │
└──────────────────────────────────┘
         ↓
Live on Production! 🚀
```

5.2 Build Configuration

Netlify Configuration (netlify.toml):

```toml
[build]
command = "npm run build"
publish = "dist"
environment = { NODE_VERSION = "18" }
```

```
[context.production]
command = "npm run build"
environment = { NODE_ENV = "production" }

[context.deploy-preview]
command = "npm run build"

[[redirects]]
from = "/*"
to = "/index.html"
status = 200
```

Build Scripts (package.json):

```
{
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview",
    "lint": "eslint src --ext .js,.jsx",
    "test": "jest",
    "e2e": "cypress open"
  }
}
```

5.3 Environment Variables

Development (.env.local):

```
VITE_API_URL=http://localhost:3000
VITE_ENV=development
```

Production (.env.production):

```
VITE_API_URL=https://api.zupharm.com
VITE_ENV=production
```

Netlify Build Variables:

```
NODE_ENV=production
NODE_VERSION=18
NPM_FLAGS=--production
```

5.4 Deployment Checklist

```
☐ Code committed to main branch
☐ All tests passing
☐ No console warnings/errors
☐ README updated (if needed)
☐ Version bumped in package.json
☐ Release notes documented
☐ Staging tested on develop
☐ Netlify build success
☐ Production health check passed
☐ Slack notification sent
☐ Analytics monitored
☐ Error tracking active
☐ User communication sent (if major)
```

5.5 Rollback Procedure

If Production Issue Occurs:

```
1. Detect Issue (within 5 mins)
   - Error monitoring alerts
   - User reports
   - Analytics anomaly

2. Immediate Response:
   - Slack #incidents channel
   - Team notified
   - Status page updated

3. Rollback Decision:
   - If critical: Revert main branch
   - If minor: Create hotfix branch
   git revert <commit-hash>
   git push origin main

4. Netlify Auto-Redeploys:
   - Previous version live
   - Cache cleared
   - Users see stable version

5. Post-Mortem:
   - Root cause analysis
   - Fix in hotfix branch
   - Re-deploy corrected code
   - Document lesson learned
```

---

6. Release Management

6.1 Release Cycle

Cadence: Weekly releases (Friday EOD)

```
Monday-Thursday: Development & QA
Friday Morning: Final testing
Friday Afternoon: Deploy to production
Monday: Monitor & gather feedback
```

6.2 Version Numbering (Semantic Versioning)

```
Format: MAJOR.MINOR.PATCH

v0.1.0 - MVP Release
v0.1.1 - Bugfix release
v0.2.0 - New features release
v1.0.0 - General Availability

MAJOR: Breaking changes
MINOR: New features (backward compatible)
PATCH: Bugfixes (backward compatible)
```

6.3 Release Notes Template

```
# Release v0.1.1 - June 10, 2026

## New Features
- ✨ Blog/Knowledge Centre page with 6 articles
- ✨ Franchise form with multi-field validation

## Bug Fixes
- 🐛 Fixed contact form email validation
- 🐛 Fixed mobile menu toggle on Safari
- 🐛 Fixed product image loading on slow connection

## Improvements
- ⚡ Optimized homepage animations (reduced motion)
- ⚡ Improved accessibility (WCAG AA compliance)
- ⚡ Updated design system documentation

## Breaking Changes
None

## Migration Guide
N/A

## Known Issues
- Blog search not yet implemented (coming in v0.2.0)

## Performance Metrics
```

```
- Bundle size: 150KB (gzip)
- Lighthouse Score: 94/100
- Core Web Vitals: All green

## Contributors
- Piyush (@piyush)
- Farhat Afreen (@farhat)
- Chhavi (@chhavi)
- Tanisha (@tanisha)

## Download
- [GitHub Release](https://github.com/Farha-t/SPRINT_Pharma/releases/v0.1.1)
```

---

7. Monitoring & Incident Response

7.1 Monitoring Stack

Tool	Purpose	Threshold
Netlify Analytics	Traffic, page views	N/A
Google Analytics	User behavior tracking	N/A
Sentry (future)	Error tracking	Alert on > 10 errors/min
Lighthouse CI (future)	Performance monitoring	Alert if score drops < 80
Uptime Robot (future)	Site availability	Alert if down > 5 mins

7.2 Incident Response Process

```
Incident Detected
        ↓
┌─────────────────────────────────┐
│ Alert Triggered                 │
├─────────────────────────────────┤
│ - Slack alert to #incidents     │
│ - Page/SMS to on-call dev       │
│ - Severity assigned: P0/P1/P2   │
└─────────────────────────────────┘
        ↓
┌─────────────────────────────────┐
│ Triage (5 mins)                 │
├─────────────────────────────────┤
│ - Gather info                   │
│ - Assess impact                 │
│ - Trigger incident lead         │
└─────────────────────────────────┘
        ↓
┌─────────────────────────────────┐
│ Mitigation (15 mins)            │
```

```
┌─────────────────────────────────────┐
│ - Identify root cause               │
│ - Apply temporary fix/rollback      │
│ - Restore service                   │
└─────────────────────────────────────┘
        ↓
┌─────────────────────────────────────┐
│ Resolution (1-4 hours)              │
├─────────────────────────────────────┤
│ - Permanent fix deployed            │
│ - Verification testing              │
│ - Team notified                     │
└─────────────────────────────────────┘
        ↓
┌─────────────────────────────────────┐
│ Post-Mortem (next day)              │
├─────────────────────────────────────┤
│ - Root cause analysis               │
│ - Preventive measures               │
│ - Documentation update              │
│ - Team learning session             │
└─────────────────────────────────────┘
```

---

8. Team Communication & Collaboration

8.1 Communication Channels

Channel	Purpose	Frequency
#general	General announcements	Daily
#development	Dev updates, help	Throughout day
#code-review	PR reviews, feedback	Throughout day
#qa-testing	QA reports, test results	Daily
#incidents	Production issues	As needed
#deployments	Deployment notifications	Weekly

8.2 Meeting Schedule

Meeting	Day	Time	Duration	Attendees
Daily Standup	M-F	10:00 AM	10 mins	All devs
Weekly Backlog Refinement	Wednesday	2:00 PM	30 mins	Product + Dev
Sprint Planning	Monday	10:30 AM	1 hour	All team
Sprint Retrospective	Friday	4:00 PM	45 mins	All team

Meeting	Day	Time	Duration	Attendees
Code Review Sync	Thursday	3:00 PM	30 mins	Devs + QA

8.3 Daily Standup Format

```
Each person (2 mins max):
1. What did I complete yesterday?
2. What am I working on today?
3. Any blockers/help needed?

Example:
"Yesterday: Finished contact form validation
Today: Working on franchise form, will PR by EOD
Blocker: Need API endpoint for form submission"
```

---

9. Documentation & Knowledge Management

9.1 Documentation Standards

Document	Location	Owner	Update Frequency
README.md	Root folder	Dev Lead	Per release
ARCHITECTURE.md	Root folder	Dev Lead	Per major change
Component Docs	Component files	Author	When updated
API Docs	/docs folder	Backend	When API changes
Deployment Guide	/docs folder	DevOps	When deploy changes

9.2 Code Comments

JSX File Example:

```jsx
/**
 * ProductCard Component
 * Displays a single product with image, name, description
 *
 * @param {string} img - Product image URL
 * @param {string} name - Product name
 * @param {string} desc - Product description
 * @returns {React.ReactElement} Rendered product card
 */
const ProductCard = ({ img, name, desc }) => {
  // Implementation
};
```

10. Performance & Optimization Standards

10.1 Performance Targets

Metric	Target	Monitoring
Page Load	< 3s	Lighthouse
Time to Interactive	< 5s	Lighthouse
Largest Contentful Paint	< 2.5s	Core Web Vitals
Cumulative Layout Shift	< 0.1	Core Web Vitals
Bundle Size	< 200KB	Webpack Bundle Analyzer

10.2 Optimization Checklist

```
Before Commit:
☐ No console warnings/errors
☐ Images optimized (compressed)
☐ No unused CSS
☐ No unused JavaScript
☐ Lighthouse audit > 90

Before Deploy:
☐ Lighthouse production > 95
☐ Mobile performance > 90
☐ Accessibility > 95
☐ Best Practices > 90
☐ SEO > 95
```

11. Security & Compliance Workflow

11.1 Security Checklist

```
Before Deployment:
☐ No hardcoded secrets (API keys, passwords)
☐ Environment variables properly configured
☐ Dependencies up-to-date (npm audit)
☐ No console.log() of sensitive data
☐ HTTPS enabled
☐ CORS properly configured
☐ Input validation on all forms
☐ XSS protection in place
☐ CSRF tokens implemented
```

11.2 Dependency Management

```
# Weekly dependency audit
npm audit

# Update dependencies safely
npm update
npm install

# Check for vulnerabilities
npm audit --fix

# Major version updates
npm outdated
npm install package@latest --save
```

---

12. Continuous Integration/Continuous Deployment (CI/CD)

12.1 CI/CD Pipeline (Planned)

```
Developer Push
    ↓
Git Webhook
    ↓
├─ Run Tests (Jest)
├─ Lint Code (ESLint)
├─ Build Check
├─ Accessibility Audit (axe-core)
├─ Performance Audit (Lighthouse)
└─ Security Scan (npm audit)
    ↓
All Checks Pass?
├─ YES → Approve merge
└─ NO → Fail build, notify dev
    ↓
Auto Deploy to Staging
    ↓
QA Testing
    ↓
Manual Approval
    ↓
Deploy to Production
    ↓
Monitor & Alert
```

---

13. Retrospective & Lessons Learned

13.1 Sprint Retrospective Template

```
Date: [Sprint End Date]
Sprint: [Sprint Number]
Duration: [2 weeks]

Attendees: [List]

1. What Went Well?
   - [Points]
   - [Points]

2. What Could Be Improved?
   - [Points]
   - [Points]

3. Action Items (Next Sprint)
   - [Person] → [Task] → [Deadline]

4. Metrics
   - Story points completed: [X]
   - Velocity: [X points/sprint]
   - Bugs found: [X]
   - Deploy frequency: [X times]
```

---

14. Workflow Process Checklist

Before Starting Development:

```
☐ Jira ticket created & assigned
☐ Acceptance criteria clear
☐ Story points estimated
☐ Pull latest from develop
☐ Create feature branch
☐ Notify team in Slack
```

During Development:

```
☐ Test locally (npm run dev)
☐ Commit regularly
☐ Push daily
☐ Check for conflicts
☐ Respond to PR comments
```

Before PR Creation:

```
☐ npm run lint passes
☐ Tested on multiple browsers
☐ Tested on mobile (375px, 768px, 1024px)
☐ No console errors
☐ Updated README (if needed)
☐ Added comments to complex code
```

Before Merge:

```
☐ PR reviewed & approved
☐ All checks pass (CI/CD)
☐ QA tested on staging
☐ No conflicts with develop
☐ Squash commits if needed
```

Before Production Deploy:

```
☐ Staging environment tested
☐ Performance acceptable
☐ Accessibility compliant
☐ Security audit passed
☐ Release notes prepared
☐ Team notified
☐ Monitoring active
```

---

15. Workflow Decision Log

Decision	Reason	Impact
2-week sprints	Balance flexibility & planning	Predictable releases
Friday deploys	Less risky than mid-week	Team available for issues
Squash commits	Clean git history	Easier to revert if needed
PR requirement	Code quality & knowledge sharing	Slight slower deployment
Automated tests	Prevent regressions	CI/CD validation

Document Version: 1.0
Last Updated: June 10, 2026
Maintained By: Farhat Afreen & Development Team
Next Review: June 30, 2026